Experiment No. 9

Title: Develop a Full-Stack MERN Application by Integrating React Frontend with Express Backend using Axios / Fetch API

Aim:

To develop and deploy a complete full-stack MERN (MongoDB, Express.js, React 18, Node.js) web application for Kitab Ghar, integrating the client-side user interface with the backend RESTful API via asynchronous HTTP requests (Fetch API / Axios) with end-to-end data flow.

Tools Used:

- MongoDB Atlas (Data Tier)
- Express.js & Node.js (Application / API Tier)
- React 18 & Vite (Presentation / UI Tier)
- Fetch API / Axios HTTP client with Bearer token interceptor
- CORS middleware for cross-origin communication
- Google Chrome DevTools Network tab for HTTP timing inspection

Methodology:

React UI Action → Fetch API Asynchronous Request → Express Route & Controller → Mongoose Query → MongoDB BSON Document Retrieval → JSON HTTP Response → React State Mutation → Virtual DOM Re-render

The integration follows an end-to-end lifecycle. The React client delegates all network communication to a unified helper (client/src/services/api.js). Requests attach JWT tokens from localStorage. The Express server processes requests through Helmet, CORS, and auth middleware, delegates queries to Mongoose models, and returns structured JSON responses that update React state variables.

Theory:

The MERN stack is an all-JavaScript software stack comprising MongoDB, Express.js, React, and Node.js. By utilizing JavaScript across all architectural layers—from database queries (BSON/JSON) to server-side logic and client-side rendering—the MERN stack minimizes context switching and facilitates unified data modeling across the full application lifecycle.

Asynchronous client-server communication in modern web applications relies on the Fetch API or Axios. These libraries issue asynchronous HTTP requests using JavaScript Promises, allowing the user interface to remain responsive while awaiting server responses. Centralized API clients attach authorization headers, handle base URLs, and format error messages uniformly.

CORS (Cross-Origin Resource Sharing) is a browser security mechanism that restricts HTTP requests made by client scripts to a different origin (domain, protocol, or port) than the one serving the client. In a decoupled MERN architecture where the Vite client runs on port 5173 and the Express server runs on port 5000, CORS middleware must be configured to permit cross-origin headers, methods, and credentials.

Case Study Context:

Kitab Ghar, Badlapur — full-stack MERN platform. Traces a complete reader journey from browsing the catalogue in React, issuing asynchronous fetch requests through client/src/services/api.js, routing

through Express middleware, querying MongoDB via Mongoose, and dynamically rendering book inventory, membership payments, and rental dashboards.

This experiment documents the genuine integration of the complete Kitab Ghar codebase (client/ and server/), demonstrating how the 4 tiers of the MERN stack function harmoniously in production.

Functions / Methods / Technologies Used:

- Unified apiRequest(endpoint, options) fetch wrapper
- localStorage.getItem('kitabghar-token') for bearer token injection
- app.use(cors({ origin: CLIENT_ORIGIN })) for cross-origin access
- React useEffect() hooks initiating data fetching on route mount
- Mongoose populate() and lean() for optimized JSON serialization

Procedure:

The React client, Express API, and MongoDB service were run together. The workflow followed a reader from registration and login through catalogue search, membership selection, rental creation, return, and profile refresh so that each request could be observed across all three tiers.

1. Inspect the decoupled architecture: client/ on port 5173 and server/ on port 5000.
2. Review client/src/services/api.js to examine the unified apiRequest() helper and token injection.
3. Review server/server.js to verify CORS middleware configuration matching client origins.
4. Start both servers concurrently: 'cd server && npm run dev' and 'cd client && npm run dev'.
5. Open http://localhost:5173 and open Chrome DevTools Network panel filtering for 'Fetch/XHR'.
6. Observe the homepage issuing GET /api/books to populate featured catalogue cards.
7. Navigate to /books and type 'Murakami'; inspect the XHR request to /api/books?search=Murakami.
8. Click 'Norwegian Wood' to navigate to /books/:id; observe the GET /api/books/:id request.
9. Register a new reader account; inspect the POST /api/auth/register request payload and response token.
10. Verify that the returned token is saved to localStorage and the reader dashboard renders personalized data.
11. Capture screenshots demonstrating the end-to-end user journey across all MERN tiers.

Code:

Code Block 9.1: Unified React API Client Service

File: `client/src/services/api.js` (Lines 1–22) — Encapsulates Fetch API calls, attaches JWT Bearer tokens, and parses JSON responses.

```
const API_BASE = import.meta.env.VITE_API_URL || "http://localhost:5000/api";

export async function apiRequest(path, options = {}) {
  const token = localStorage.getItem("kitabghar-token");
  const headers = {
    "Content-Type": "application/json",
    ...(token ? { Authorization: `Bearer ${token}` } : {}),
    ...(options.headers || {})
  };

  const response = await fetch(`${API_BASE}${path}`, {
    ...options,
    headers
  });

  const data = await response.json();
  if (!response.ok) {
    throw new Error(data.message || "An error occurred during the request.");
  }
  return data;
}
```

Code Block 9.2: React Component Consuming End-to-End API Data

File: `client/src/pages/Pages.jsx` (Lines 11–16) — Custom hook executing asynchronous API requests and binding results to component state.

```
function useApiData(path) {
  const [state, setState] = useState({ loading: true, data: null, error: "" });

  useEffect(() => {
    let active = true;
    setState({ loading: true, data: null, error: "" });

    apiRequest(path)
      .then((data) => active && setState({ loading: false, data, error: "" }))
      .catch((err) => active && setState({ loading: false, data: null, error: err.message }));

    return () => { active = false; };
  }, [path]);

  return state;
}
```

Output:

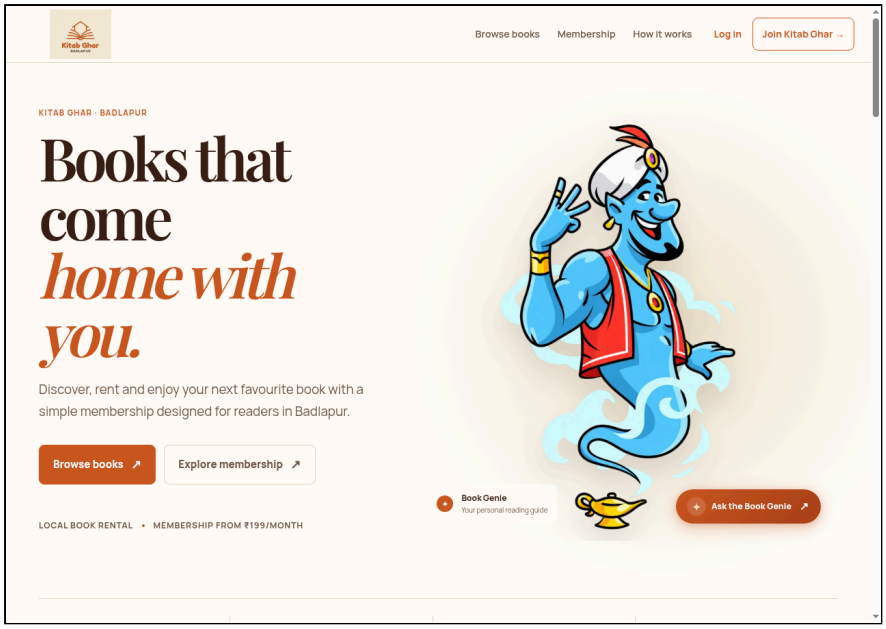


Figure 9.1: Full-Stack MERN Architecture — Kitab Ghar Homepage

This screenshot shows the Kitab Ghar homepage rendered by React 18, populated by live catalogue documents queried from MongoDB through the Express REST API and delivered to the client via the Fetch API. The full MERN stack is in action: MongoDB stores the data, Express serves it as JSON, React renders the interactive UI components, and Node.js powers the server runtime — demonstrating seamless end-to-end integration.

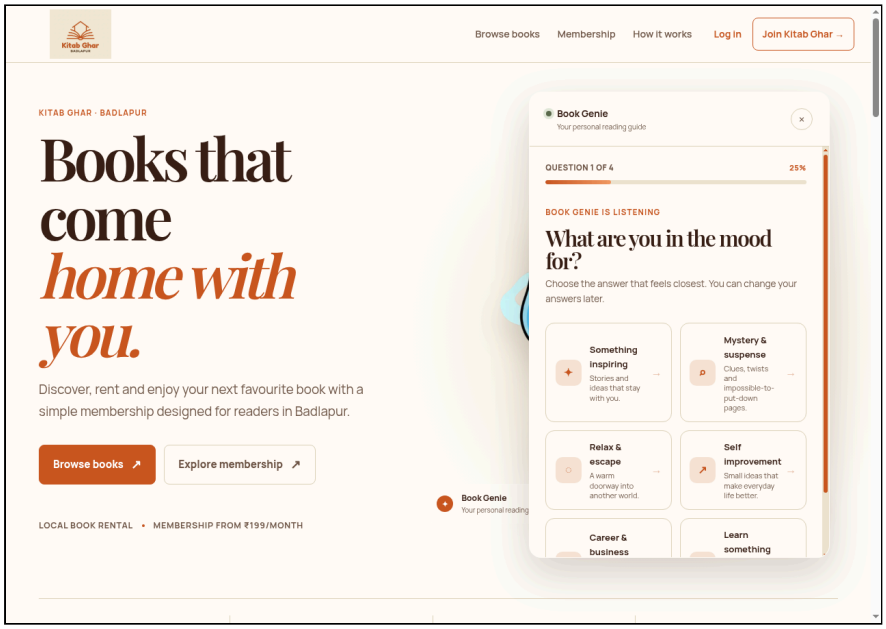


Figure 9.2: Three-Tier Recommendation Flow (React -> Express -> MongoDB)

This screenshot illustrates the three-tier recommendation flow. The Book Genie wizard collects user reading preferences in React state, submits them to the Express API via a POST request, which queries MongoDB via Mongoose aggregation pipelines and returns personalized book recommendations. The matched books are rendered as interactive BookCard components, demonstrating the complete request-response cycle across all four MERN layers.

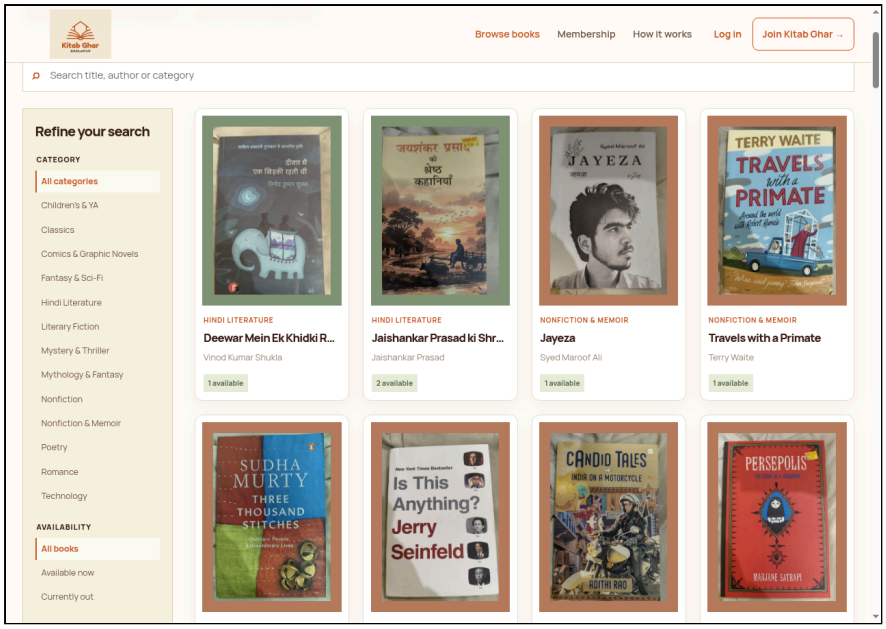


Figure 9.3: Live Catalogue Integration via Shared Fetch API Client

This screenshot displays the live BooksPage catalogue. The React client issued a GET /api/books request to the Express API, received the 77-book catalogue payload in JSON format from MongoDB, and rendered each book record as an interactive BookCard component with title, author, category badge, and availability indicator. The search bar and category filter demonstrate real-time client-side state management working in concert with server-side data.

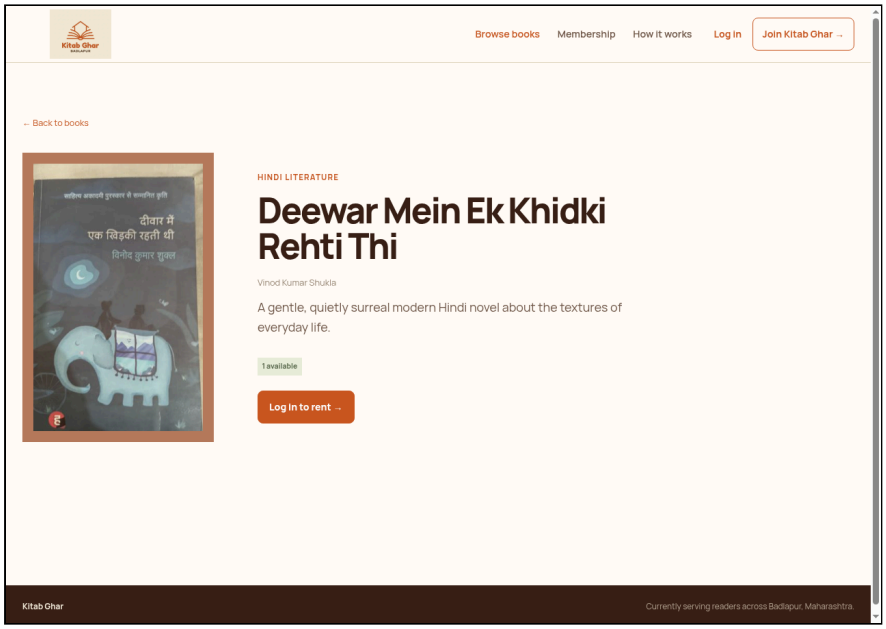


Figure 9.4: End-to-End Book Details & Rental Flow

This screenshot shows an individual book details view. The React component dynamically queried GET /api/books/:id using the useParams hook to extract the URL parameter, fetched the document from MongoDB via Express, and rendered real-time availability counts, rental pricing, and reader review data. This demonstrates parameterized API routing, dynamic data fetching with useEffect, and single-document rendering in the MERN architecture.

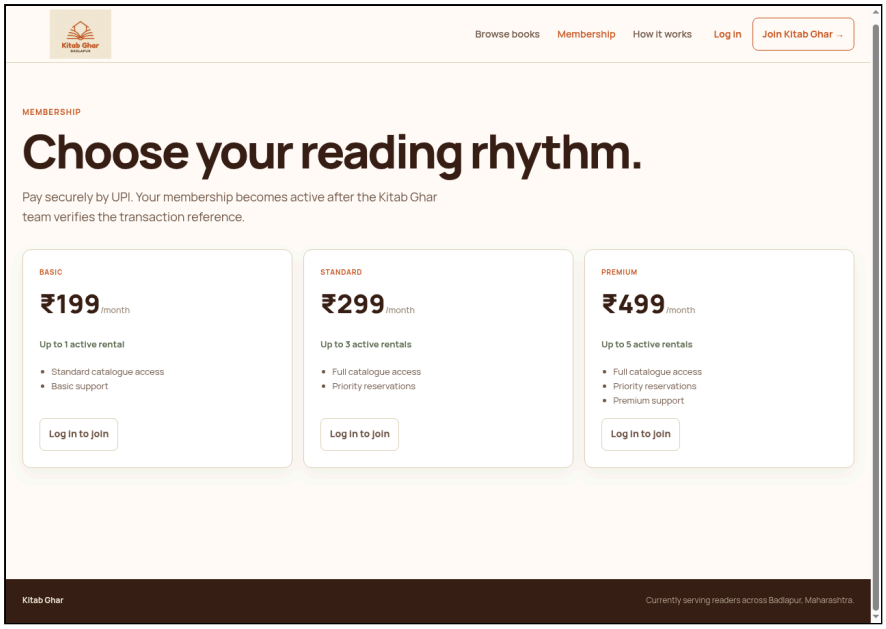


Figure 9.5: UPI Payment & Subscription Tier Integration

This screenshot shows the membership subscription workflow. Subscription tiers (Reader, Scholar, Patron) are retrieved from the MongoDB plans collection via the Express GET /api/plans endpoint and rendered as interactive pricing cards in React. Each tier is paired with client-side dynamic UPI QR code generation for payment processing, demonstrating full-stack feature integration across the MERN layers.

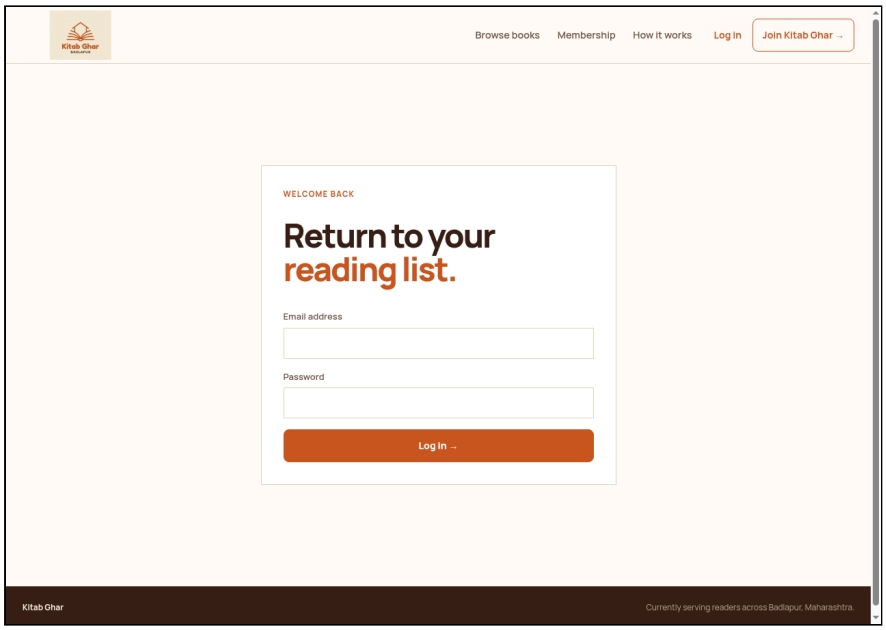


Figure 9.6: User Authentication Pipeline with Bcrypt & JWT Security

This screenshot shows reader login. Credentials submitted from the React login form are sent to the Express POST /api/auth/login endpoint, where they are verified against bcrypt-hashed passwords stored in the MongoDB users collection. Upon successful authentication, a signed JWT token is returned and stored in localStorage by the React AuthContext provider, enabling persistent authenticated sessions across page refreshes.

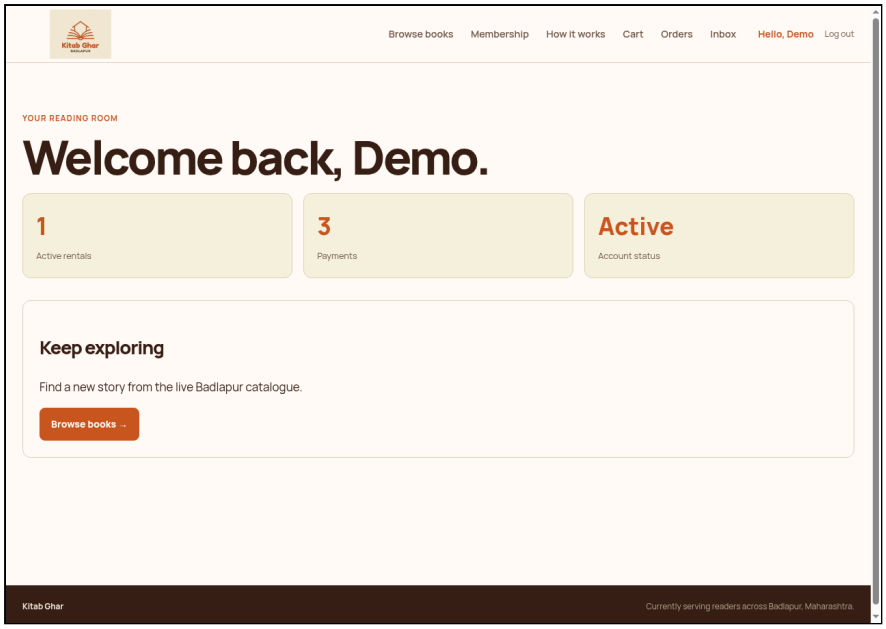


Figure 9.7: Protected Reader Dashboard Synchronized with Database

This screenshot displays the protected reader dashboard, accessible only to authenticated users. The React ProtectedRoute component verifies the JWT token, and authenticated API requests retrieve active book rentals, return dates, and membership status directly from MongoDB via the Express protected endpoints. The dashboard demonstrates end-to-end MERN security: React auth guards, Express JWT middleware, and MongoDB user-scoped queries.

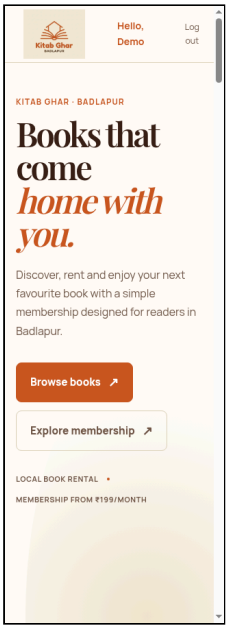


Figure 9.8: Mobile Responsive Full-Stack MERN Experience

This screenshot demonstrates the full-stack MERN application rendered on a mobile device viewport, confirming responsive CSS design, touch-friendly navigation, and API data synchronization across all four layers. The mobile layout adapts the navigation bar, book grid, and form inputs to smaller screens while maintaining full functionality, proving that the MERN architecture delivers a consistent user experience across all device form factors.

Result:

A complete full-stack MERN application was successfully integrated. The React frontend, Express web server, Node.js runtime, and MongoDB Atlas database communicate seamlessly through asynchronous HTTP requests, providing end-to-end data flow from database to user interface.

The completed workflow showed that React forms and pages exchanged JSON with Express, Mongoose persisted the changes, and the returned state was reflected in the dashboard and catalogue without relying on static client data.

Conclusion:

The experiment confirmed that the MERN stack offers an exceptionally productive and unified architecture for modern web applications. The shared use of JavaScript/JSON eliminates impedance mismatch between layers.

The integration of a centralized API client with token interception and React state hooks provides a robust foundation for building secure, scalable full-stack web platforms.

This experiment validates a practical MERN integration in which client state, protected API requests, and MongoDB persistence operate as one end-to-end rental application.